Brewing a cup of MOCCa: Hephaestos

W. Ryssens*
Institut d'Astronomie et d'Astrophysique, Université Libre de Bruxelles, Campus de la Plaine CP 226, 1050 Brussels, Belgium

M. Bender
IPNL, Université de Lyon, Université Lyon 1, CNRS/IN2P3, F-69622 Villeurbanne, France

P.-H. Heenen
PNTPM, CP229, Université Libre de Bruxelles, B-1050 Bruxelles, Belgium
(Dated: April 2022)

I. DIRECTORY AND FILE STRUCTURE OF THE ENTIRE PACKAGE

```
Core functionality
-------------------
Makefile          : contains all instructions for building working Tantalus executables.
Hephaestos.py     : main driver for Hephaestos
configs/          : contains predefined configuration files
functionals/      : contains predefined functional files
parameterizations/: contains parameterization files
exec/             : contains executables
mod/, obj/        : compilation directories for the compiler

src_heph/         : Hephaestos source code
src_orig/         : Tantalus source code, i.e. FORTRAN templates
src/              : "completed" Tantalus source code, output of Hephaestos

Optional/useful stuff
----------------------
build_all.sh      : script to compile ALL possible configurations of Tantalus
compilation_logs/ : compilation logs, associated with build_all.sh
gen_XYZ_version.sh: scripts to generate public versions of the code, for
                    distribution to various groups.
tantalus_examples : many examples for bugtesting and illustration
manual            : contains Tantalus and Hephaestos manuals
auxiliary/        : auxiliary scripts for analysing results

Not-so-useful stuff
----------------------
tex/              : Folders for future interfacing with LateX documents.
tex_templates/    : Folders for future interfacing with LateX documents.
```

* wryssens@ulb.be

II. RUNNING HEPHAESTOS / PRACTICALLY BUILDING TANTALUS

Hephaestos is a Python-3 code[1] that takes Tantalus source code templates and, depending on user choices, builds compileable FORTRAN code. Running Hephaestos is as simple as

```
python Hephaestos.py config
```

where config is the name of a configuration file from the configs/ folder (witthout the trailing '.py'). If everything went smoothly, you should now have a set of compileable Tantalus source code in the src/ folder that reflects the choices made in the configuration file.
In day-to-day practice, I use the Makefile instead of running Hephaestos manually. By typing

```
make CONFIG=config
```

which (i) runs Hephaestos and (ii) starts compiling immediately. Other optional arguments for the make command are

```
CXX = compiler    => ifort/gfortran work by default)
PRE = ""          => If this is given, Hephaestos is NOT run before starting the
                     compilation process.
```

At the end of a successful compilation process, you will find a new executable in the exec/ folder.

III. CONFIGURATION FILES

The configuration files contain the settings with respect to symmetry choices and advanced options regarding EDFs. The default configuration file for example contains the following instructions (I've removed non-relevant comments for brevity):

```
FUNC_FILE = 'NLO.func'           ! functional file to use for compilation

SYMSTRING = 'Rz,T,P,STy'         ! Symmetries to conserve in the calculation
REDUCE    = [1,1,1]              ! Reduce (=1) or not (=0) every Cartesian axis
                                 ! [1,1,1] : calculations in 1/8th of the box
                                 ! [1,1,0] : use complete z-axis, EV4-style
                                 ! Note: the code will ensure coherence between
                                 ! SYMSTRING and this input.

INSYM     = 'Rz,T,P,STy'         ! Same options as before: these now refer to
INREDUCE  = [1,1,1]              ! what type of .wf files this executable will
                                 ! be able to read.

QUANT_AXIS='Z'                   ! Orientation options for the definition of
SECOND_AXIS=1                    ! multipole moments.

PH_PP_DECOUPL = True             ! If True: all feedback through density-dependencies
                                 ! of the pairing interaction will be ignored.
                                 ! For example: a standard ULB-delta-interaction
                                 ! depends on D_I_I, but its contribution to
                                 ! F_I_I is ignored.
```

[1] Running with Python2.7 or before will likely fail.

IV. FUNCTIONAL FILES

Functional files are the true input to Hephaestos; they contain the specification of the EDF. They consist of three parts, that can be described schematically as

- Description part: lines starting with '#'. These serve to describe the functional and are copied into Hephaestos output. They have no effect on the FORTRAN code generated. Ends with the last line starting with '#'.
- Parameters part : a list of parameters of the functional. These are the quantities that Hephaestos will assume will be read from a parameterization file. This is the place to put the t_i, x_i and more parameters. Ends when the user includes a line '!TERMS'.
- Terms part: dedicated description of terms, will be explained below.

In between, it is possible to write human-readable comments starting with '!'. These will be ignored completely by both Hephaestos and Tantalus. The only exception is the specific '!TERMS', which is used to indicate the transition between the second and third part of a .func file.
A functional file in the end looks like this, the (abbreviated) content of the default functional file.

```
# Standard NLO Skyrme functional                                                    | Description
# -> with time-odd terms                                                            | Description
# -> without tensor terms                                                           | Description
# -> Jmunu and sDs terms optional                                                   | Description
! Particle-hole parameters                                                          | Comment
t0 ;  x0 ; t1 ;  x1 ; t2 ; x2 ; t3; x3; t3b; x3b;  wso ; wsoq ; sigma ; sigma_b     | Parameters
! Pairing parameters                                                               | Comment
Vn ; Vp ; alpha ; sigmap  ; CT0 ; CT1 ; CSDS0 ; CSDS1                              | Parameters
!TERMS                                                                            | TRANSITION
!-----------------------------------------------------------------------------------| Comment
! Central, LO terms                                                                | Comment
! Time-even                                                                        | Comment
! Term                        ! Coupling constant           ! DD_pow! isospin     | Comment
E_D_I_I_D_I_I                 ; Cc(t0,x0,+1,0,0)            ;   1    ; 0 ; 0      | Term
E_D_I_I_D_I_I                 ; Cc(t0,x0,+1,0,1)            ;   1    ; 1 ; 1      | Term
[.......]
E_D_I_I_CP_I_I_CP_I_I         ; -Vn/4*alpha/(0.16**sigmap)  ; sigmap; 0 ; n ; n   | Term
E_D_I_I_CP_I_I_CP_I_I         ; -Vp/4*alpha/(0.16**sigmap)  ; sigmap; 0 ; p ; p   | Term
```

To include a term in the functional, one needs to specify

- Its structure.
- Its coupling constant
- Whether or not it is density dependent
- The isospin components of all densities involved.

The simplest term

$$C_0^{\rho\rho}\rho_0(\mathbf{r})\rho_0(\mathbf{r}) \tag{1}$$

is encoded as

```
E_D_I_I_D_I_I                 ; Cc(t0,x0,+1,0,0)            ;   1    ; 0 ; 0
```

where

- Different columns are separated by ';'
- The first column tells the code about $\rho\rho = D^{I,I}D^{I,I}$. Note the underscores separating (i) the different operators 'I' in the densities and (ii) the different densities and (iii) the initial 'E'. These are all mandatory.
- The expression `Cc(t0,x0,+1,0,0)` is the coupling constant. This can be any string, but Hephaestos will simply paste this into the relevant parts of Tantalus. Hence, this string needs to be valid FORTRAN code; you can include function calls (as I do here), but these need to be defined in the FORTRAN code. You can freely use any parameters included in the first part of the func file. Note that the convention of the code for coupling constants is the most logical one: i.e. all possible signs and factors should be included in this column. I.e. the code will program in this case simply $E = CcD^{I,I}D^{I,I}$.

- The '1' indicates the power of the density dependence of the first density in the term. In case this is '1', the code will ignore the possibilities of this term being density-dependent.
- The final columns indicate the isospin components: 0 or 1 for particle-hole terms and 'p' or 'n' for pairing densities. Note that the code expects one index for every density; tri/quadrilinear terms need three/four indices.

In general, the code can deal with terms T of the form

$$T = (X_{t_1}^{\hat{L}_1,\hat{R}_1})^{\alpha} \prod_{i=2}^{i_{\max}} X_{t_i}^{\hat{L}_i,\hat{R}_i} \,. \tag{2}$$

where $i_{\max} = 2, 3, 4$ depending if we are dealing with a bi-, tri- or quadrilinear term, X can stand for $D, C, \tilde{D}$ or $\tilde{C}$ and the isospin indices t_i can all be independent . Hephaestos is powerful, but is limited by the following for the moment:

- Only the first density in the product can be raised to a power.
- Only one vector-product $\mathbf{x} \times \mathbf{y}$ between indices can be present in a single density.
- All spin-indices/operators in all densities should be part of the right-operators, i.e. in $\hat{R}_i$, never in the $\hat{L}_i$.

Furthermore, I want to emphasize that **Hephaestos is not so clever as he seems and checks for consistency of your input only in very limited ways**. It is **EXTREMELY EASY** to give Hephaestos wrong input that either (i) crashes Hephaestos with an unintelligible error warning (ii) exits gracefully, but produces uncompileable FORTRAN code or, even worse, (iii) generates a correct Tantalus executable that is subtly wrong in an unforeseen way.

A. The structure of densities

I'll use examples to explain the writing of any given density for Hephaestos:

$$D^{I,I} \rightarrow \mathsf{D\_I\_I} \tag{3}$$
$$D^{(\nabla,\nabla)} \rightarrow \mathsf{D\_Nm\_Nm} \tag{4}$$
$$C^{I,\nabla\sigma} \rightarrow \mathsf{C\_I\_NS} \tag{5}$$
$$\tilde{D}^{I,I} \rightarrow \mathsf{DP\_I\_I}\,. \tag{6}$$

In short:

- Replace ∇ by 'N' and σ by 'S'.
- Indicate contraction by repeated summation indices like 'm'. Other valid options are currently 'k', 'q', 'o', 'l'.
- Indicate pairing densities by adding a 'P' after the 'D' or 'C'.

To indicate external derivatives, use prefixes Der_ and Lap_, such as for example

$$\Delta D^{1,1} \rightarrow \mathsf{Lap\_D\_I\_I} \tag{7}$$
$$\sum_{\mu\nu\kappa} \nabla_\mu \epsilon_{\mu\nu\kappa} C_{\nu\kappa}^{I,\nabla\sigma} \rightarrow \mathsf{Derm\_C\_I\_NxmSxm}\,. \tag{8}$$

This last example is perfect to demonstrate the useage of vector products: indicate the two last indices of the Levi-civita symbol by x, then link these two with the third summation index by repeated small summation letters (here m).

B. The structure of terms

Terms in Hephaestos are just combinations of densities linked by underscores and preceded by E_. Of course, this means that contractions and/or vector products can involve multiple densities. The code has no problems with

`E_D_I_I_Derm_C_I_NxmSxm or E_C_Nm_NkSo_C_Nm_NkSo`

which are (i) the time-even spin-orbit term of NLO functionals and (ii) the $(\mathrm{Im}T_{\mu\nu\kappa})^2$ terms of the Becker N2LO formulation.

C. Some remarks

- The code figures out what densities and contractions to calculate from your input.
- It time-reversal is broken is however, the cranking parts of the code require that you put at least one term with $C^{I,\nabla}$ and one with $D^{I,\sigma}$. Their coupling constants can be zer.
- You are yourself in charge of making sure your terms are time-reversal invariant.
- If time-reversal is conserved, the code is smart enough to drop all time-odd terms it finds.